ensure that the team rapidly builds what the product owner wants, the product owner actively collaborates with the ScrumMaster and development team and must be available to answer questions soon after they are posed. See Chapter 9 for a detailed description of the product owner role.

### ScrumMaster

The ScrumMaster helps everyone involved understand and embrace the Scrum values, principles, and practices. She acts as a coach, providing process leadership and helping the Scrum team and the rest of the organization develop their own high-performance, organization-specific Scrum approach. At the same time, the ScrumMaster helps the organization through the challenging change management process that can occur during a Scrum adoption.

As a facilitator, the ScrumMaster helps the team resolve issues and make improvements to its use of Scrum. She is also responsible for protecting the team from outside interference and takes a leadership role in removing **impediments** that inhibit team productivity (when the individuals themselves cannot reasonably resolve them). The ScrumMaster has no authority to exert control over the team, so this role is not the same as the traditional role of project manager or development manager. The ScrumMaster functions as a leader, not a manager. I will discuss the roles of functional manager and project manager in Chapter 13. See Chapter 10 for more details on the ScrumMaster role.

### Development Team

Traditional software development approaches discuss various job types, such as architect, programmer, tester, database administrator, UI designer, and so on. Scrum defines the role of a development team, which is simply a diverse, cross-functional collection of these types of people who are responsible for designing, building, and testing the desired product.

The development team self-organizes to determine the best way to accomplish the goal set out by the product owner. The development team is typically five to nine people in size; its members must collectively have all of the skills needed to produce good-quality, working software. Of course, Scrum can be used on development efforts that require much larger teams. However, rather than having one Scrum team with, say, 35 people, there would more likely be four or more Scrum teams, each with a development team of nine or fewer people. See Chapter 11 for more details on the development team role and Chapter 12 for more details on coordinating multiple teams.

## Scrum Activities and Artifacts

Figure 2.3 illustrates most of the Scrum activities and artifacts and how they fit together.

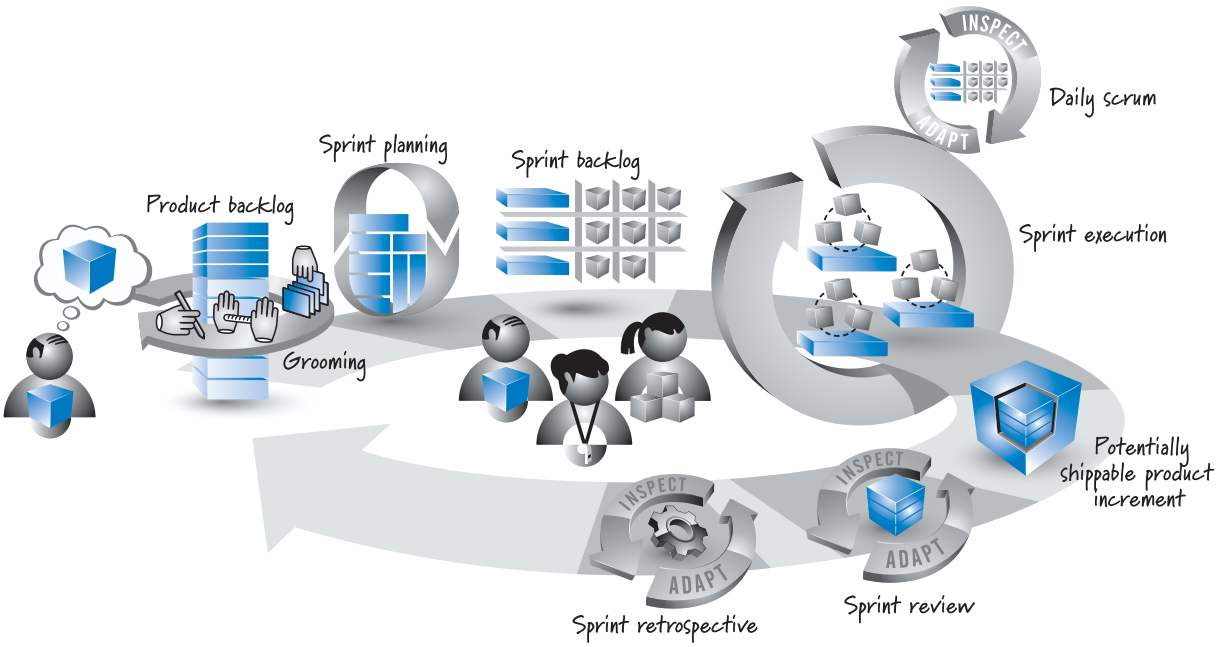



**FIGURE 2.3** Scrum framework

Let's summarize the diagram, starting on the left side of the figure and working clockwise around the main looping arrow (the sprint).

The product owner has a vision of what he wants to create (the big cube). Because the cube can be large, through an activity called **grooming** it is broken down into a set of features that are collected into a prioritized list called the product backlog.

A sprint starts with sprint planning, encompasses the development work during the sprint (called sprint execution), and ends with the review and retrospective. The sprint is represented by the large, looping arrow that dominates the center of the figure. The number of items in the product backlog is likely to be more than a development team can complete in a short-duration sprint. For that reason, at the beginning of each sprint, the development team must determine a subset of the product backlog items it believes it can complete—an activity called sprint planning, shown just to the right of the large product backlog cube.

As a brief aside, in 2011 a change in "The Scrum Guide" (Schwaber and Sutherland 2011) generated debate about whether the appropriate term for describing the result of sprint planning is a **forecast** or a **commitment**. Advocates of the word *forecast* like it because they feel that although the development team is making the best estimate that it can at the time, the estimate might change as more information becomes known during the course of the sprint. Some also believe that a commitment on the part of the team will cause the team to sacrifice quality to meet the commitment or will cause the team to "under-commit" to guarantee that the commitment is met.

I agree that all development teams should generate a forecast (estimate) of what they can deliver each sprint. However, many development teams would benefit from

using the forecast to derive a commitment. Commitments support mutual trust between the product owner and the development team as well as within the development team. Also, commitments support reasonable short-term planning and decision making within an organization. And, when performing multiteam product development, commitments support synchronized planning—one team can make decisions based on what another team has committed to do. In this book, I favor the term *commitment*; however, I occasionally use *forecast* if it seems correct in context.

To acquire confidence that the development team has made a reasonable commitment, the team members create a second backlog during sprint planning, called the sprint backlog. The sprint backlog describes, through a set of detailed **tasks**, how the team plans to design, build, integrate, and test the selected subset of features from the product backlog during that particular sprint.

Next is sprint execution, where the development team performs the tasks necessary to realize the selected features. Each day during sprint execution, the team members help manage the flow of work by conducting a synchronization, inspection, and adaptive planning activity known as the daily scrum. At the end of sprint execution the team has produced a potentially shippable product increment that represents some, but not all, of the product owner's vision.

The Scrum team completes the sprint by performing two inspect-and-adapt activities. In the first, called the sprint review, the stakeholders and Scrum team inspect the product being built. In the second, called the sprint retrospective, the Scrum team inspects the Scrum process being used to create the product. The outcome of these activities might be adaptations that will make their way into the product backlog or be included as part of the team's development process.

At this point the Scrum sprint cycle repeats, beginning anew with the development team determining the next most important set of product backlog items it can complete. After an appropriate number of sprints have been completed, the product owner's vision will be realized and the solution can be released.

In the remainder of this chapter I will discuss each of these activities and artifacts in greater detail.

## Product Backlog

Using Scrum, we always do the most valuable work first. The product owner, with input from the rest of the Scrum team and stakeholders, is ultimately responsible for determining and managing the sequence of this work and communicating it in the form of a prioritized (or ordered) list known as the **product backlog** (see Figure 2.4). On new-product development the product backlog items initially are features required to meet the product owner's vision. For ongoing product development, the product backlog might also contain new features, changes to existing features, defects needing repair, technical improvements, and so on.

The product owner collaborates with internal and external stakeholders to gather and define the product backlog items. He then ensures that product backlog items

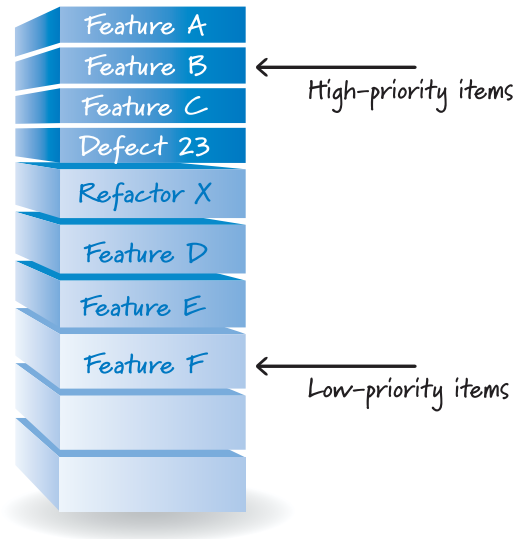



**FIGURE 2.4** Product backlog

are placed in the correct sequence (using factors such as value, cost, knowledge, and risk) so that the high-value items appear at the top of the product backlog and the lower-value items appear toward the bottom. The product backlog is a constantly evolving artifact. Items can be added, deleted, and revised by the product owner as business conditions change, or as the Scrum team's understanding of the product grows (through feedback on the software produced during each sprint).

Overall the activity of creating and refining product backlog items, estimating them, and prioritizing them is known as grooming (see Figure 2.5).

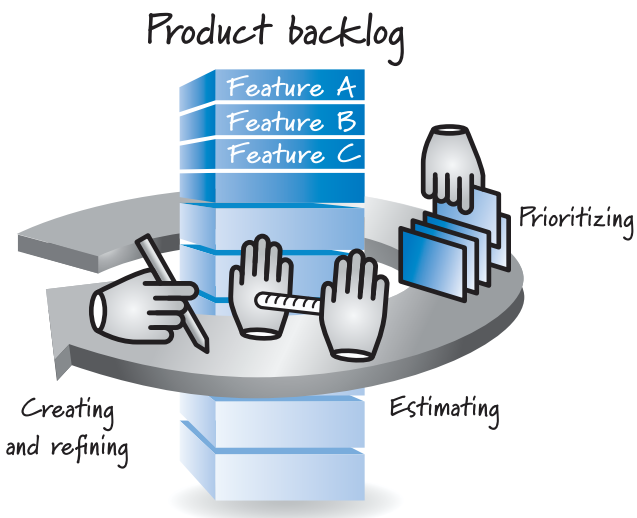



**FIGURE 2.5** Product backlog grooming

As a second brief aside, in 2011 there was another debate as to whether the appropriate term for describing the sequence of items in the product backlog should be *prioritized* (the original term) or *ordered*, the term used in "The Scrum Guide" (Schwaber and Sutherland 2011). The argument was that prioritizing is simply one form of ordering (and, according to some, not even the most appropriate form of ordering). The issue of how to best sequence items in the product backlog, however, is influenced by many factors, and a single word may never capture the full breadth and depth of the concept. Although there may be theoretical merit to the ordered-versus-prioritized debate, most people (including me) use the terms interchangeably when discussing the items in the product backlog.

Before we finalize prioritizing, ordering, or otherwise arranging the product backlog, we need to know the size of each item in the product backlog (see Figure 2.6).

Size equates to cost, and product owners need to know an item's cost to properly determine its priority. Scrum does not dictate which, if any, size measure to use with product backlog items. In practice, many teams use a **relative size measure** such as **story points** or **ideal days**. A relative size measure expresses the overall size of an item in such a way that the absolute value is not considered, but the relative size of an item compared to other items is considered. For example, in Figure 2.6, feature C is size 2 and feature E is size 8. What we can conclude is that feature E is about four times larger than feature C. I will discuss these measures further in Chapter 7.

## Sprints

In Scrum, work is performed in iterations or cycles of up to a calendar month called **sprints** (see Figure 2.7). The work completed in each sprint should create something of tangible value to the customer or user.

Sprints are **timeboxed** so they always have a fixed start and end date, and generally they should all be of the same duration. A new sprint immediately follows the completion of the previous sprint. As a rule we do not permit any goal-altering changes in scope or personnel during a sprint; however, business needs sometimes make adherence to this rule impossible. I will describe sprints in more detail in Chapter 4.

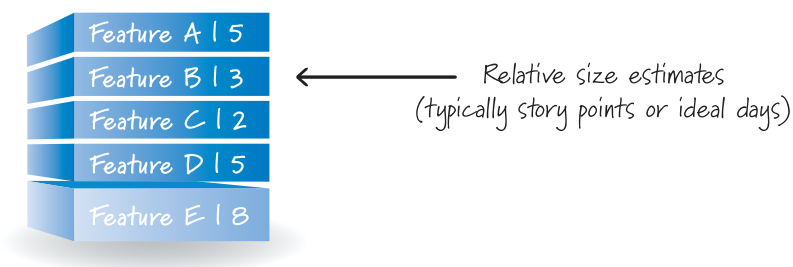



**FIGURE 2.6** Product backlog item sizes

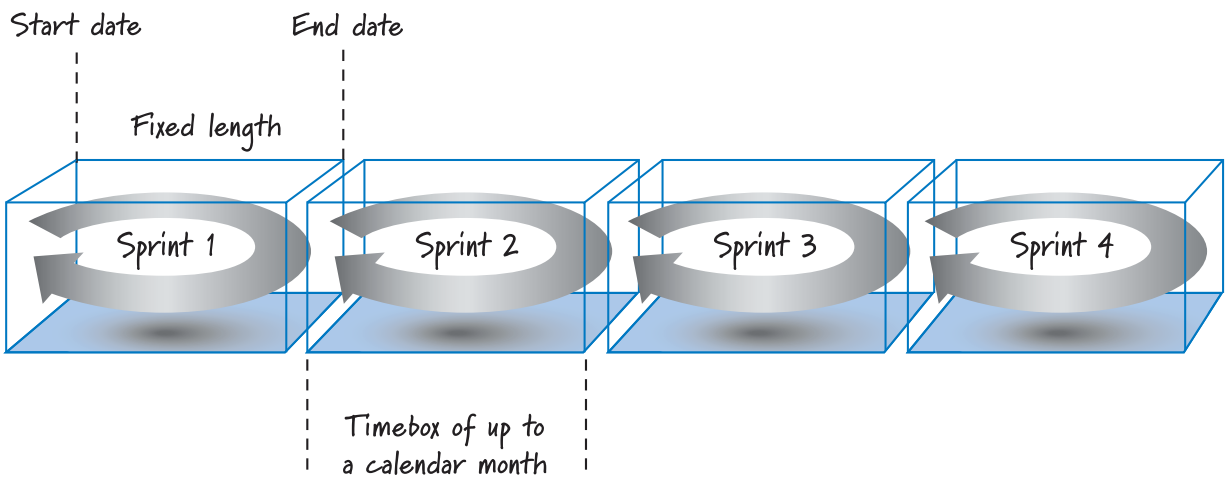



**FIGURE 2.7** Sprint characteristics

## Sprint Planning

A product backlog may represent many weeks or months of work, which is much more than can be completed in a single, short sprint. To determine the most important subset of product backlog items to build in the next sprint, the product owner, development team, and ScrumMaster perform **sprint planning** (see Figure 2.8).

During sprint planning, the product owner and development team agree on a **sprint goal** that defines what the upcoming sprint is supposed to achieve. Using this

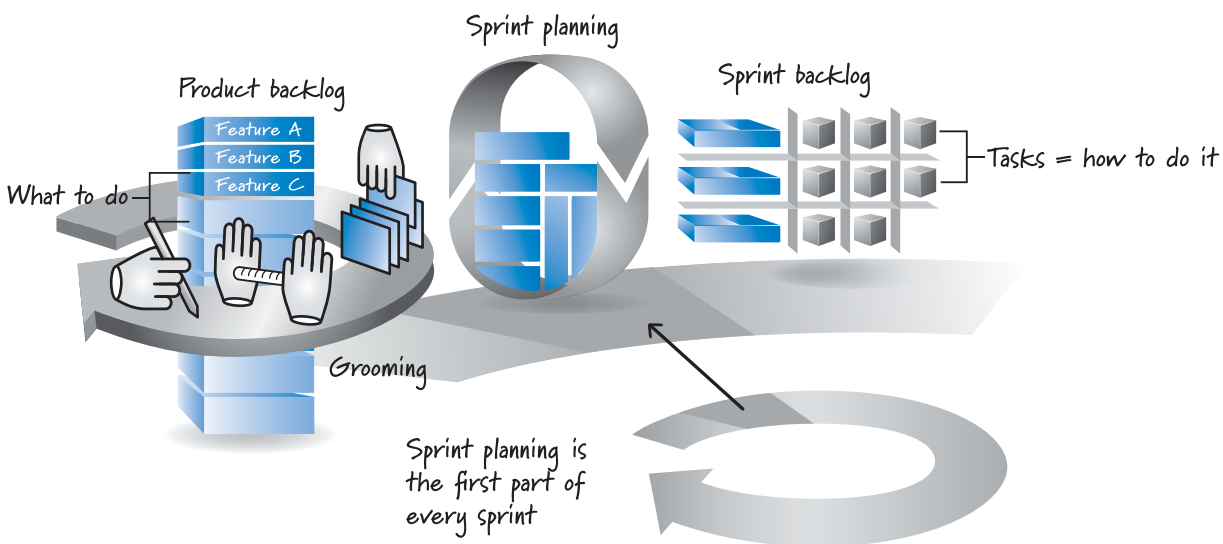



**FIGURE 2.8** Sprint planning

goal, the development team reviews the product backlog and determines the high-priority items that the team can realistically accomplish in the upcoming sprint while working at a **sustainable pace**—a pace at which the development team can comfortably work for an extended period of time.

To acquire confidence in what it can get done, many development teams break down each targeted feature into a set of tasks. The collection of these tasks, along with their associated product backlog items, forms a second backlog called the **sprint backlog** (see Figure 2.9).

The development team then provides an estimate (typically in hours) of the effort required to complete each task. Breaking product backlog items into tasks is a form of design and **just-in-time** planning for how to get the features done.

Most Scrum teams performing sprints of two weeks to a month in duration try to complete sprint planning in about four to eight hours. A one-week sprint should take no more than a couple of hours to plan (and probably less). During this time there are several approaches that can be used. The approach I use most often follows a simple cycle: Select a product backlog item (whenever possible, the next-most-important item as defined by the product owner), break the item down into tasks, and determine if the selected item will reasonably fit within the sprint (in combination with other items targeted for the same sprint). If it does fit and there is more capacity to complete work, repeat the cycle until the team is out of capacity to do any more work.

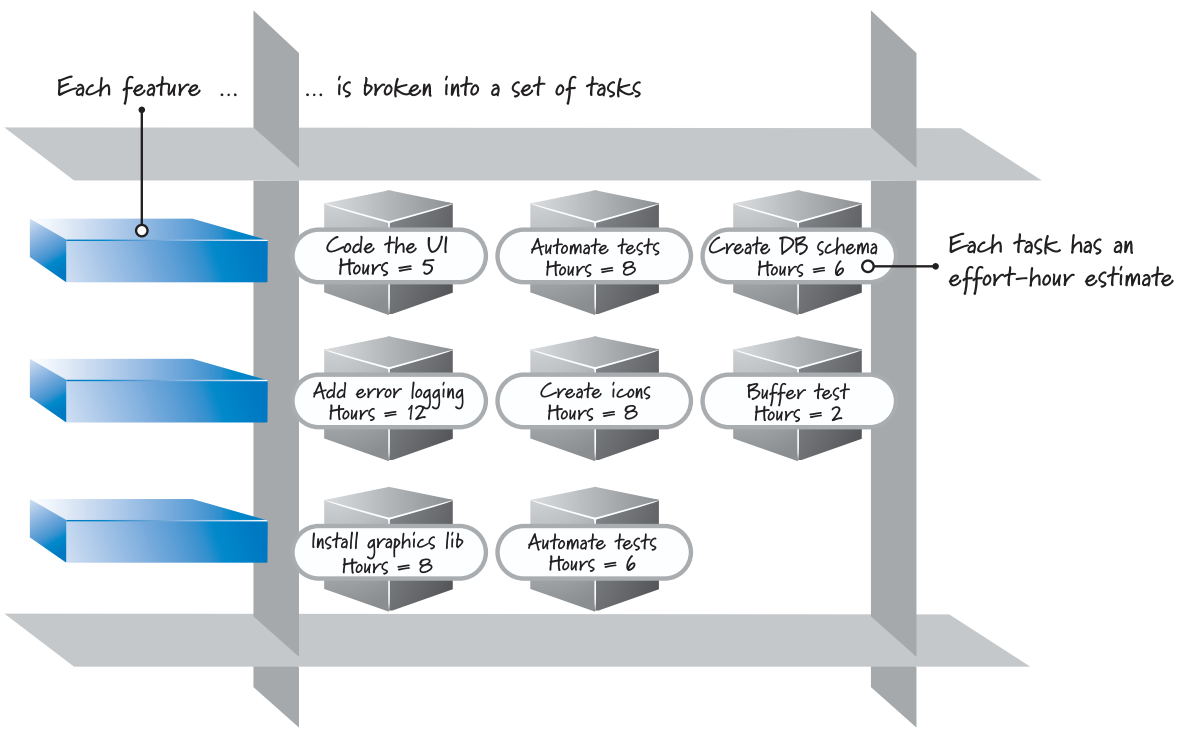



**FIGURE 2.9** Sprint backlog

An alternative approach would be for the product owner and team to select all of the target product backlog items at one time. The development team alone does the task breakdowns to confirm that it really can deliver all of the selected product backlog items. I will describe each approach in more detail in Chapter 19.

## Sprint Execution

Once the Scrum team finishes sprint planning and agrees on the content of the next sprint, the development team, guided by the ScrumMaster's coaching, performs all of the task-level work necessary to get the features done (see Figure 2.10), where "done" means there is a high degree of confidence that all of the work necessary for producing good-quality features has been completed.

Exactly what tasks the team performs depends of course on the nature of the work (for example, are we building software and what type of software, or are we building hardware, or is this marketing work?).

Nobody tells the development team in what order or how to do the task-level work in the sprint backlog. Instead, team members define their own task-level work and then self-organize in any manner they feel is best for achieving the sprint goal. See Chapter 20 for more details on sprint execution.

## Daily Scrum

Each day of the sprint, ideally at the same time, the development team members hold a timeboxed (15 minutes or less) **daily scrum** (see Figure 2.11). This inspect-and-adapt activity is sometimes referred to as the **daily stand-up** because of the common practice of everyone standing up during the meeting to help promote brevity.

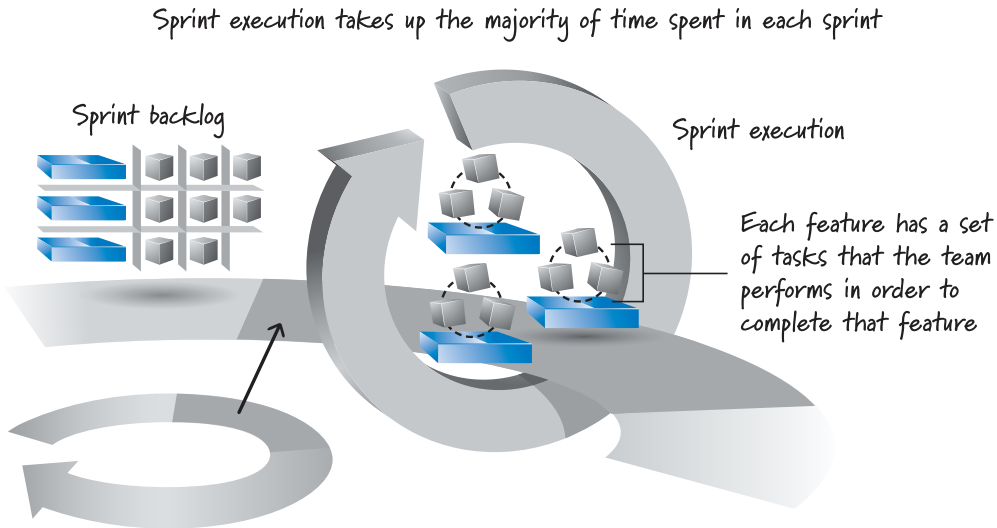



**FIGURE 2.10** Sprint execution

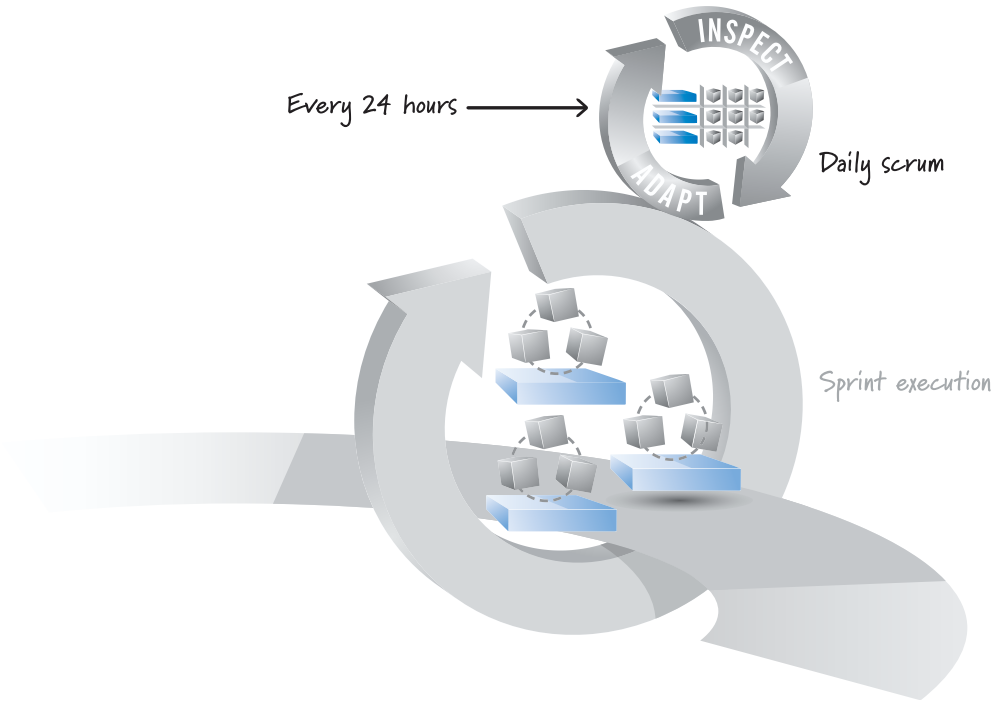



**FIGURE 2.11** Daily scrum

A common approach to performing the daily scrum has the ScrumMaster facilitating and each team member taking turns answering three questions for the benefit of the other team members:

- What did I accomplish since the last daily scrum?
- What do I plan to work on by the next daily scrum?
- What are the obstacles or impediments that are preventing me from making progress?

By answering these questions, everyone understands the big picture of what is occurring, how they are progressing toward the sprint goal, any modifications they want to make to their plans for the upcoming day's work, and what issues need to be addressed. The daily scrum is essential for helping the development team manage the fast, flexible flow of work within a sprint.

The daily scrum is not a problem-solving activity. Rather, many teams decide to talk about problems after the daily scrum and do so with a small group of interested people. The daily scrum also is not a traditional status meeting, especially the kind historically called by project managers so that they can get an update on the project's status. A daily scrum, however, can be useful to communicate the status of sprint backlog items among the development team members. Mainly, the daily scrum is an inspection, synchronization, and adaptive daily planning activity that helps a self-organizing team do its job better.

Although their use has fallen out of favor, Scrum has used the terms "**pigs**" and "**chickens**" to distinguish who should participate during the daily scrum versus who simply observes. The farm animals were borrowed from an old joke (which has several variants): "In a ham-and-eggs breakfast, the chicken is involved, but the pig is committed." Obviously the intent of using these terms in Scrum is to distinguish between those who are involved (the chickens) and those who are committed to meeting the sprint goal (the pigs). At the daily scrum, only the pigs should talk; the chickens, if any, should attend as observers.

I have found it most useful to consider everyone on the Scrum team a pig and anyone who isn't, a chicken. Not everyone agrees. For example, the product owner is not required to be at the daily scrum, so some consider him to be a chicken (the logic being, how can you be "committed" if you aren't required to attend?). This seems wrong to me, because I can't imagine how the product owner, as a member of the Scrum team, is any less committed to the outcome of a sprint than the development team. The metaphor of pigs and chickens breaks down if you try to apply it within a Scrum team.

## Done

In Scrum, we refer to the sprint results as a **potentially shippable product increment** (see Figure 2.12), meaning that whatever the Scrum team agreed to do is really done according to its agreed-upon definition of done. This definition specifies the degree

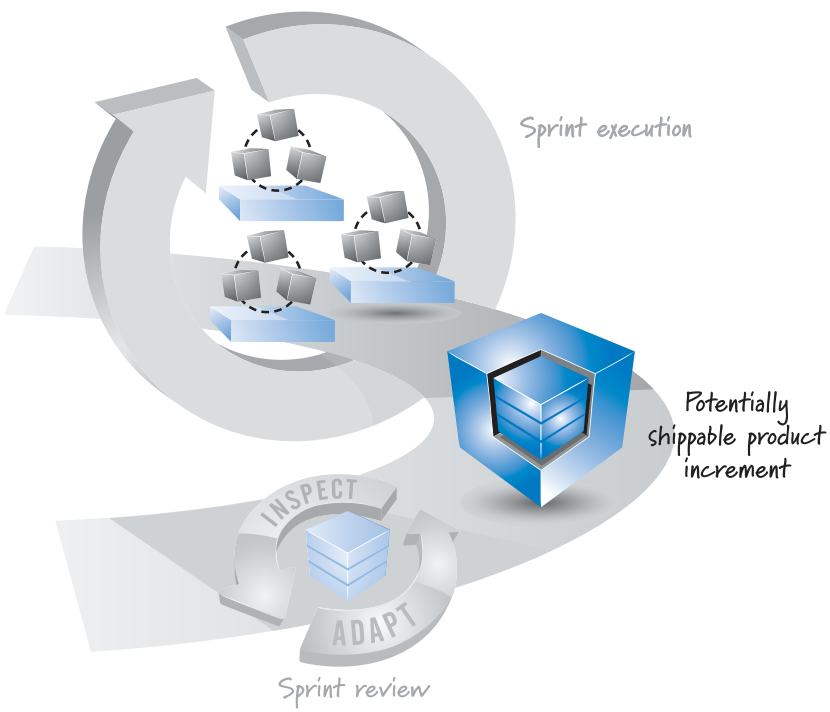



**FIGURE 2.12** Sprint results (potentially shippable product increment)

of confidence that the work completed is of good quality and is potentially shippable. For example, when developing software, a bare-minimum definition of done should yield a complete slice of product functionality that is designed, built, integrated, tested, and documented.

An aggressive definition of done enables the business to decide each sprint if it wants to ship (or deploy or release) what got built to internal or external customers.

To be clear, "potentially shippable" does not mean that what got built must actually be shipped. Shipping is a business decision, which is frequently influenced by things such as "Do we have enough features or enough of a customer workflow to justify a customer deployment?" or "Can our customers absorb another change given that we just gave them a release two weeks ago?"

Potentially shippable is better thought of as a state of confidence that what got built in the sprint is actually done, meaning that there isn't materially important undone work (such as important testing or integration and so on) that needs to be completed before we can ship the results from the sprint, if shipping is our business desire.

As a practical matter, over time some teams may vary the definition of done. For example, in the early stages of game development, having features that are potentially shippable might not be economically feasible or desirable (given the exploratory nature of early game development). In these situations, an appropriate definition of done might be a slice of product functionality that is sufficiently functional and usable to generate feedback that enables the team to decide what work should be done next or how to do it. See Chapter 4 for more details on the definition of done.

## Sprint Review

At the end of the sprint there are two additional inspect-and-adapt activities. One is called the **sprint review** (see Figure 2.13).

The goal of this activity is to inspect and adapt the product that is being built. Critical to this activity is the conversation that takes place among its participants, which include the Scrum team, stakeholders, sponsors, customers, and interested members of other teams. The conversation is focused on reviewing the just-completed features in the context of the overall development effort. Everyone in attendance gets clear visibility into what is occurring and has an opportunity to help guide the forthcoming development to ensure that the most business-appropriate solution is created.

A successful review results in bidirectional information flow. The people who aren't on the Scrum team get to sync up on the development effort and help guide its direction. At the same time, the Scrum team members gain a deeper appreciation for the business and marketing side of their product by getting frequent feedback on the convergence of the product toward delighted customers or users. The sprint review therefore represents a scheduled opportunity to inspect and adapt the product. As a

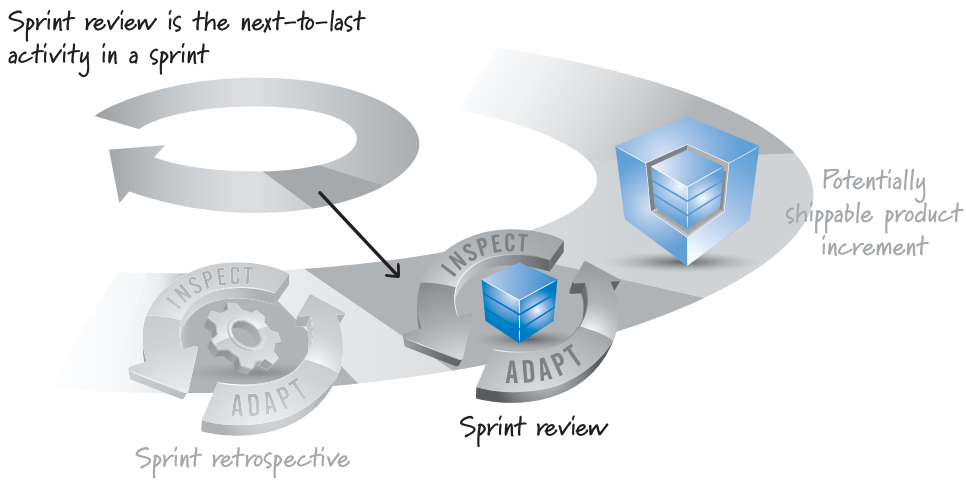



**FIGURE 2.13** Sprint review

matter of practice, people outside the Scrum team can perform intra-sprint feature reviews and provide feedback to help the Scrum team better achieve its sprint goal. See Chapter 21 for more details on the sprint review.

## Sprint Retrospective

The second inspect-and-adapt activity at the end of the sprint is the **sprint retrospective** (see Figure 2.14). This activity frequently occurs after the sprint review and before the next sprint planning.

Whereas the sprint review is a time to inspect and adapt the product, the sprint retrospective is an opportunity to inspect and adapt the process. During the sprint retrospective the development team, ScrumMaster, and product owner come together

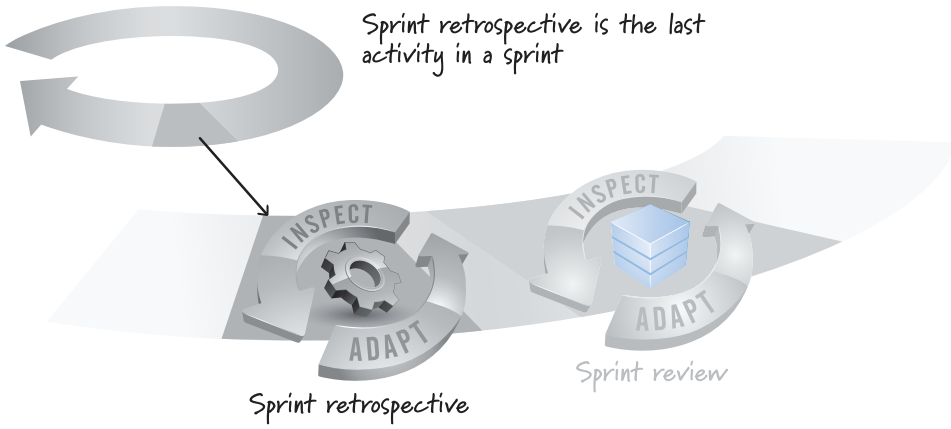



**FIGURE 2.14** Sprint retrospective

to discuss what is and is not working with Scrum and associated technical practices. The focus is on the continuous process improvement necessary to help a good Scrum team become great. At the end of a sprint retrospective the Scrum team should have identified and committed to a practical number of process improvement actions that will be undertaken by the Scrum team in the next sprint. See Chapter 22 for details on the sprint retrospective.

After the sprint retrospective is completed, the whole cycle is repeated again—starting with the next sprint-planning session, held to determine the current highest-value set of work for the team to focus on.

## Closing

This chapter described core Scrum practices, focusing on an end-to-end description of the Scrum framework's roles, activities, and artifacts. There are other practices, such as higher-level planning and progress-tracking practices, that many Scrum teams use. These will be described in subsequent chapters. In the next chapter, I will provide a description of the core principles on which Scrum is based. This will facilitate the deeper exploration of the Scrum framework in subsequent chapters.

# Chapter 3
# Agile Principles

Before we delve deeper into the mechanics of Scrum, it will be helpful to understand the underlying principles that drive and inform those mechanics.

This chapter describes the agile principles that underlie Scrum and compares them with those of traditional, plan-driven, sequential product development. In doing so, the chapter sets the stage for understanding how Scrum differs from more traditional forms of product development and for a more detailed analysis of Scrum practices in subsequent chapters.

## Overview

I find it instructive to introduce Scrum's underlying principles by comparing them with the beliefs that drive more traditional, plan-driven, sequential development. Doing so makes it easier for people to understand how Scrum is similar to or different from something they know and understand.

The goal of comparing agile principles with traditional development principles is not to make the case that plan-driven, sequential development is bad and that Scrum is good. Both are tools in the professional developer's toolkit; there is no such thing as a bad tool, rather just inappropriate times to use that tool. As I described briefly in the context of the Cynefin framework in Chapter 1, Scrum and traditional, plan-driven, sequential development are appropriate to use on different classes of problems.

In making the comparison between the two approaches, I am using the pure or "textbook" description of plan-driven, sequential development. By taking this perspective when describing traditional development, I am better able to draw out the distinctions and more clearly illustrate the principles that underlie Scrum-based development.

One pure form of traditional, plan-driven development frequently goes by the term **waterfall** (see Figure 3.1). However, that is just one example of a broader class of **plan-driven** processes (also known as **traditional**, **sequential**, **anticipatory**, **predictive**, or **prescriptive development processes**).

Plan-driven processes are so named because they attempt to plan for and anticipate up front all of the features a user might want in the end product, and to determine how best to build those features. The idea here is that the better the planning, the better the understanding, and therefore the better the execution. Plan-driven processes are often called sequential processes because practitioners perform, in sequence, a complete requirements analysis followed by a complete design followed in turn by coding/building and then testing.

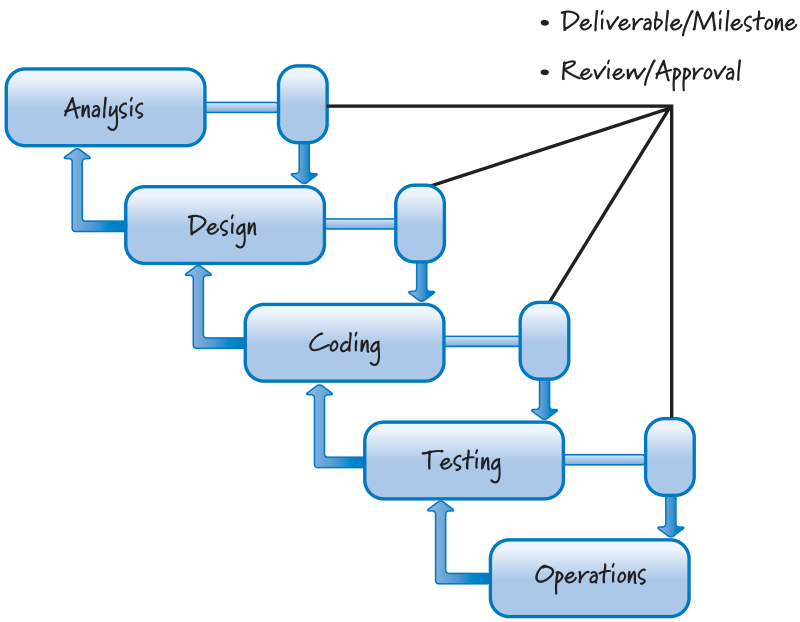



**FIGURE 3.1** Waterfall process

Plan-driven development works well if you are applying it to problems that are well defined, predictable, and unlikely to undergo any significant change. The problem is that most product development efforts are anything but predictable, especially at the beginning. So, while a plan-driven process gives the impression of an orderly, accountable, and measurable approach, that impression can lead to a false sense of security. After all, developing a product rarely goes as planned.

For many, a plan-driven, sequential process just makes sense, understand it, design it, code it, test it, and deploy it, all according to a well-defined, prescribed plan. There is a belief that it *should* work. If applying a plan-driven approach doesn't work, the prevailing attitude is that we must have done something wrong. Even if a plan-driven process repeatedly produces disappointing results, many organizations continue to apply the same approach, sure that if they just do it better, their results will improve. The problem, however, is not with the execution. It's that plan-driven approaches are based on a set of beliefs that do not match the uncertainty inherent in most product development efforts.

Scrum, on the other hand, is based on a different set of beliefs—ones that do map well to problems with enough uncertainty to make high levels of predictability difficult. The principles that I describe in this chapter are drawn from a number of sources, including the Agile Manifesto (Beck et al. 2001), lean product development (Reinertsten 2009b; Poppendieck and Poppendieck 2003), and "The Scrum Guide" (Schwaber and Sutherland 2011).

These principles are organized into several categories as shown in Figure 3.2.

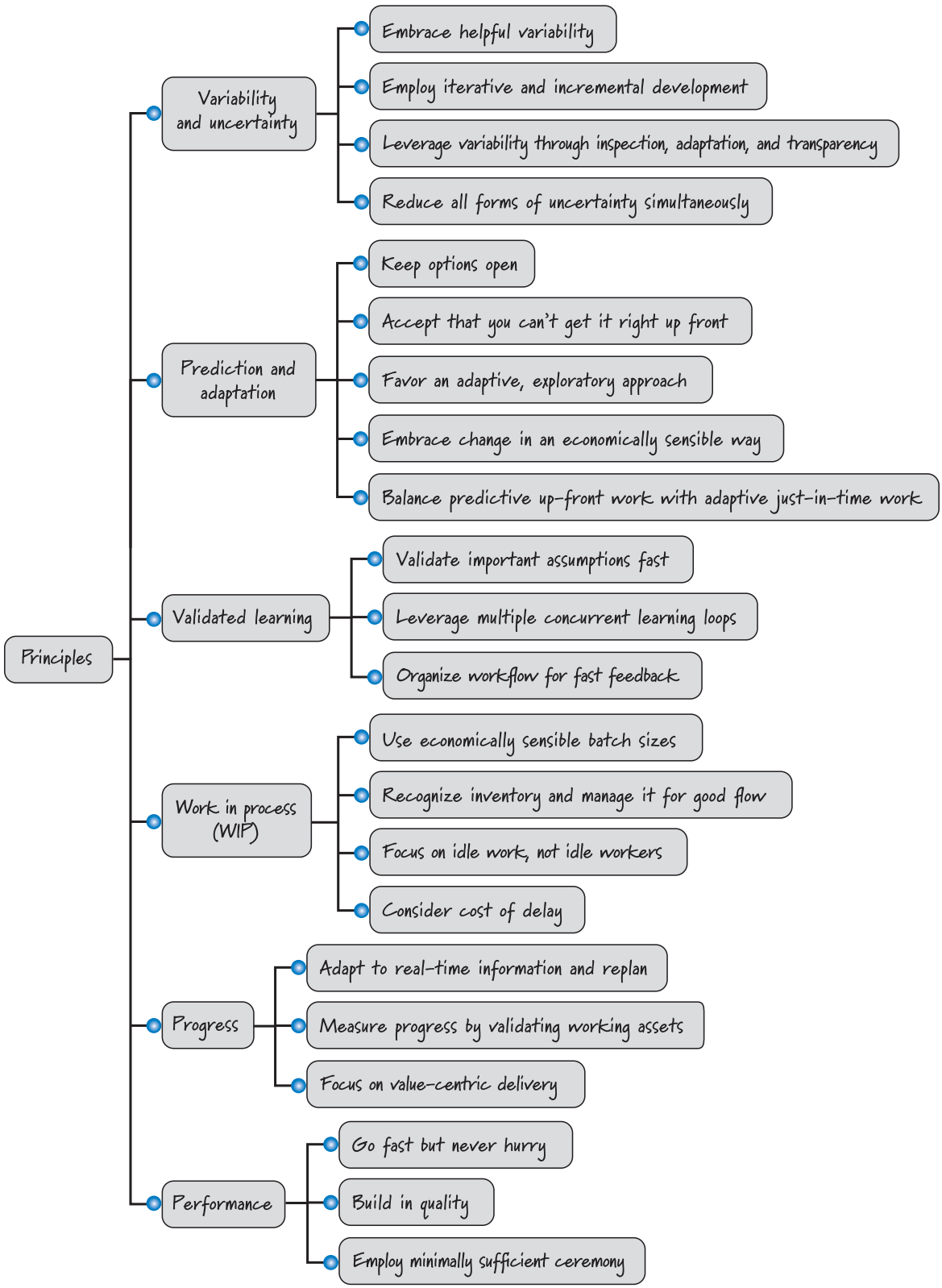



**FIGURE 3.2** Categorization of principles

I start by discussing principles that leverage the variability and uncertainty inherent in product development. This is followed by a discussion of principles that deal with balancing up-front prediction with just-in-time adaptation. Then, I discuss principles focused on learning, followed by principles for managing the work in process. I conclude by focusing on progress and performance principles.

# Variability and Uncertainty

Scrum leverages the **variability** and **uncertainty** in product development to create innovative solutions. I describe four principles related to this topic:

- Embrace helpful variability.
- Employ iterative and incremental development.
- Leverage variability through inspection, adaptation, and transparency.
- Reduce all forms of uncertainty simultaneously.

## Embrace Helpful Variability

Plan-driven processes treat product development like manufacturing—they shun variability and encourage conformance to a **defined process**. The problem is that product development is not at all like product manufacturing. In manufacturing our goal is to take a fixed set of requirements and follow a sequential set of well-understood steps to manufacture a finished product that is the same (within a defined variance range) every time (see Figure 3.3).

In product development, however, the goal is to create the unique *single instance* of the product, not to *manufacture* the product. This single instance is analogous to a unique recipe. We don't want to create the same recipe twice; if we do, we have wasted our money. Instead, we want to create a unique recipe for a new product. Some amount of variability is necessary to produce a different product each time. In fact, every feature we build within a product is different from every other feature within that product, so we need variability even at this level. Only once we have the recipe do we manufacture the product—in the case of software products, as easily as copying bits.

That being said, some manufacturing concepts do apply to product development and can and should be leveraged. For example, as I will discuss shortly, recognizing and managing **inventory** (or work in process), which is essential to manufacturing, is

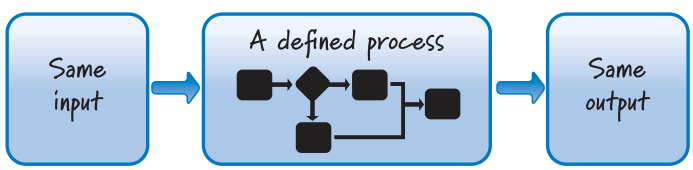



**FIGURE 3.3** Defined process

also essential in product development. By the very nature of the work involved, however, product development and product manufacturing are not at all the same thing and as such require very different processes.

## Employ Iterative and Incremental Development

Plan-driven, sequential development assumes that we will get things right up front and that most or all of the product pieces will come together late in the effort.

Scrum, on the other hand, is based on **iterative and incremental development**. Although these two terms are frequently used as if they were a single concept, iterative development is actually distinct from incremental development.

**Iterative development** acknowledges that we will probably get things wrong before we get them right and that we will do things poorly before we do them well (Goldberg and Rubin 1995). As such, iterative development is a planned rework strategy. We use multiple passes to improve what we are building so that we can converge on a good solution. For example, we might start by creating a prototype to acquire important knowledge about a poorly known piece of the product. Then we might create a revised version that is somewhat better, which might in turn be followed by a pretty good version. In the course of writing this book, for example, I wrote and rewrote each of the chapters several times as I received feedback and as my understanding of how I wanted to communicate a topic improved.

Iterative development is an excellent way to improve the product as it is being developed. The biggest downside to iterative development is that in the presence of uncertainty it can be difficult up front to determine (plan) how many improvement passes will be necessary.

**Incremental development** is based on the age-old principle of "Build some of it before you build all of it." We avoid having one large, *big-bang-style* event at the end of development where all the pieces come together and the entire product is delivered. Instead, we break the product into smaller pieces so that we can build some of it, learn how each piece is to survive in the environment in which it must exist, adapt based on what we learn, and then build more of it. While writing this book, I wrote a chapter at a time and sent each chapter out for review as it was completed, rather than trying to receive feedback on the entire book at once. This gave me the opportunity to incorporate that feedback into future chapters, adjusting my tone, style, or delivery as needed. It also gave me the opportunity to learn incrementally and apply what I learned from earlier chapters to later chapters.

Incremental development gives us important information that allows us to adapt our development effort and to change how we proceed. The biggest drawback to incremental development is that by building in pieces, we risk missing the big picture (we see the trees but not the forest).

Scrum leverages the benefits of both iterative and incremental development, while negating the disadvantages of using them individually. Scrum does this by using both ideas in an adaptive series of timeboxed iterations called sprints (see Figure 3.4).

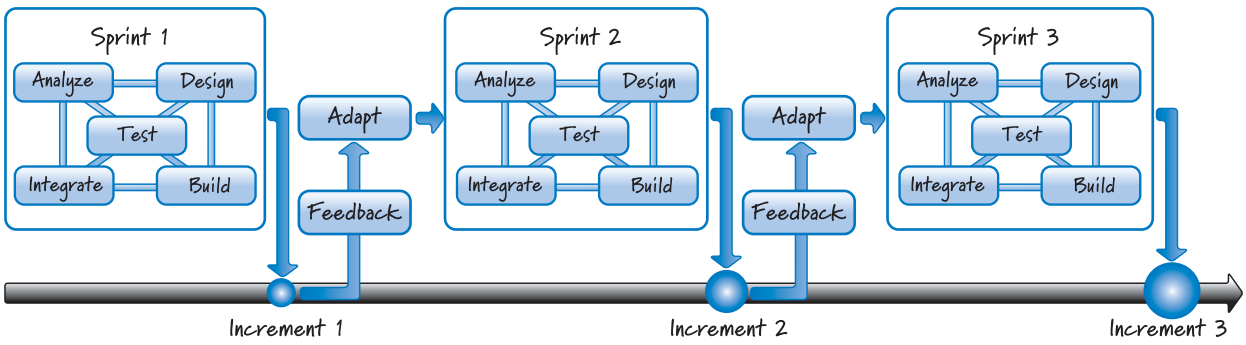



**FIGURE 3.4** Scrum uses iterative and incremental development.

During each sprint we perform all of the activities necessary to create a working product increment (some of the product, not all of it). This is illustrated in Figure 3.4 by showing that some analysis, design, build, integration, and test work is completed in each sprint. This all-at-once approach has the benefit of quickly validating the assumptions that are made when developing product features. For example, we make some design decisions, create some code based on those decisions, and then test the design and code—all in the same sprint. By doing all of the related work within one sprint, we are able to quickly rework features, thus achieving the benefits of iterative development, without having to specifically plan for additional iterations.

A misuse of the sprint concept is to focus each sprint on just one type of work—for example, sprint 1 (analysis), sprint 2 (design), sprint 3 (coding), and sprint 4 (testing). Such an approach attempts to overlay Scrum with a waterfall-style work breakdown structure. I often refer to this misguided approach as **WaterScrum**, and I have heard others refer to it as **Scrummerfall**.

In Scrum, we don't work on a phase at a time; we work on a feature at a time. So, by the end of a sprint we have created a valuable product increment (some but not all of the product features). That increment includes or is integrated and tested with any previously developed features; otherwise, it is not considered done. For example, increment 2 in Figure 3.4 includes the features of increment 1. At the end of the sprint, we can get feedback on the newly completed features within the context of already completed features. This helps us view the product from more of a big-picture perspective than we might otherwise have.

We receive feedback on the sprint results, which allows us to adapt. We can choose different features to work on in the next sprint or alter the process we will use to build the next set of features. In some cases, we might learn that the increment, though it technically fits the bill, isn't as good as it could be. When that happens, we can schedule rework for a future sprint as part of our commitment to iterative development and continuous improvement. This helps overcome the issue of not knowing

up front exactly how many improvement passes we will need. Scrum does not require that we predetermine a set number of iterations. The continuous stream of feedback will guide us to do the appropriate and economically sensible number of iterations while developing the product incrementally.

## Leverage Variability through Inspection, Adaptation, and Transparency

Plan-driven processes and Scrum are fundamentally different along several dimensions (see Table 3.1, based on dimensions suggested by Reinertsen 2009a).

A plan-driven, sequential development process assumes little or no output variability. It follows a well-defined set of steps and uses only small amounts of feedback late in the process. In contrast, Scrum embraces the fact that in product development, some level of variability is required in order to build something new. Scrum also assumes that the process necessary to create the product is complex and therefore would defy a complete up-front definition. Furthermore, it generates early and frequent feedback to ensure that the right product is built and that the product is built right.

At the heart of Scrum are the principles of **inspection**, **adaptation,** and **transparency** (referred to collectively by Schwaber and Beedle 2001 as **empirical process control**). In Scrum, we inspect and adapt not only what we are building but also how we are building it (see Figure 3.5).

To do this well, we rely on transparency: all of the information that is important to producing a product must be available to the people involved in creating the product. Transparency makes inspection possible, which is needed for adaptation. Transparency also allows everyone concerned to observe and understand what is happening. It leads to more communication and it establishes trust (both in the process and among team members).

**TABLE 3.1** Comparison of Plan-Driven and Scrum Processes

| Dimension | Plan-Driven | Scrum |
|---|---|---|
| Degree of process definition | Well-defined set of sequential steps | Complex process that would defy a complete up-front definition |
| Randomness of output | Little or no output variability | Expect variability because we are not trying to build the same thing over and over |
| Amount of feedback used | Little and late | Frequent and early |

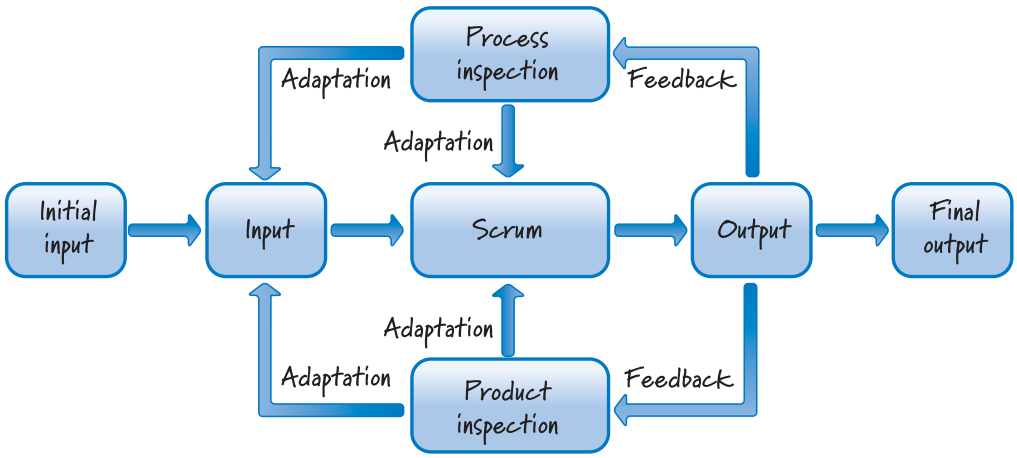



**FIGURE 3.5** Scrum process model

## Reduce All Forms of Uncertainty Simultaneously

Developing new products is a complex endeavor with a high degree of uncertainty. That uncertainty can be divided into two broad categories (Laufer 1996):

- **End uncertainty** (*what* uncertainty)—uncertainty surrounding the features of the final product
- **Means uncertainty** (*how* uncertainty)—uncertainty surrounding the process and technologies used to develop a product

In particular environments or with particular products there might also be **customer uncertainty** (*who* uncertainty). For example, start-up organizations (including large organizations that focus on novel products) may only have assumptions as to who the actual customers of their products will be. This uncertainty must be addressed or they might build brilliant products for the wrong markets.

Traditional, sequential development processes focus first on eliminating all end uncertainty by fully defining up front what is to be built, and only then addressing means uncertainty.

This simplistic, linear approach to uncertainty reduction is ill suited to the complex domain of product development, where our actions and the environment in which we operate mutually constrain one another. For example:

- We decide to build a feature (our action).
- We then show that feature to a customer, who, once he sees it, changes his mind about what he really wants, or realizes that he did not adequately convey the details of the feature (our action elicits a response from the environment).

- We make design changes based on the feedback (the environment's reaction influences us to take another unforeseen action).

In Scrum, we do not constrain ourselves by fully addressing one type of uncertainty before we address the next type. Instead, we take a more holistic approach and focus on simultaneously reducing all uncertainties (end, means, customer, and so on). Of course, at any point in time we might focus more on one type of uncertainty than another. Simultaneously addressing multiple types of uncertainty is facilitated by iterative and incremental development and guided by constant inspection, adaptation, and transparency. Such an approach allows us to opportunistically probe and explore our environment to identify and learn about the **unknown unknowns** (the things that we don't yet know that we don't know) as they emerge.

## Prediction and Adaptation

When using Scrum, we are constantly balancing the desire for prediction with the need for adaptation. I describe five agile principles related to this topic:

- Keep options open.
- Accept that you can't get it right up front.
- Favor an adaptive, exploratory approach.
- Embrace change in an economically sensible way.
- Balance predictive up-front work with adaptive just-in-time work.

### Keep Options Open

Plan-driven, sequential development requires that important decisions in areas like requirements or design be made, reviewed, and approved within their respective phases. Furthermore, these decisions must be made before we can transition to the next phase, even if those decisions are based on limited knowledge.

Scrum contends that we should never make a premature decision just because a generic process would dictate that now is the appointed time to make one. Instead, when using Scrum, we favor a strategy of keeping our options open. Often this principle is referred to as the **last responsible moment** (**LRM**) (Poppendieck and Poppendieck 2003), meaning that we delay commitment and do not make important and irreversible decisions until the last responsible moment. And when is that? When the cost of not making a decision becomes greater than the cost of making a decision (see Figure 3.6). At that moment, we make the decision.

To appreciate this principle, consider this. On the first day of a product development effort we have the least information about what we are doing. On each subsequent day of the development effort, we learn a little more. Why, then, would we ever choose to make all of the most critical, and perhaps irreversible, decisions on the first

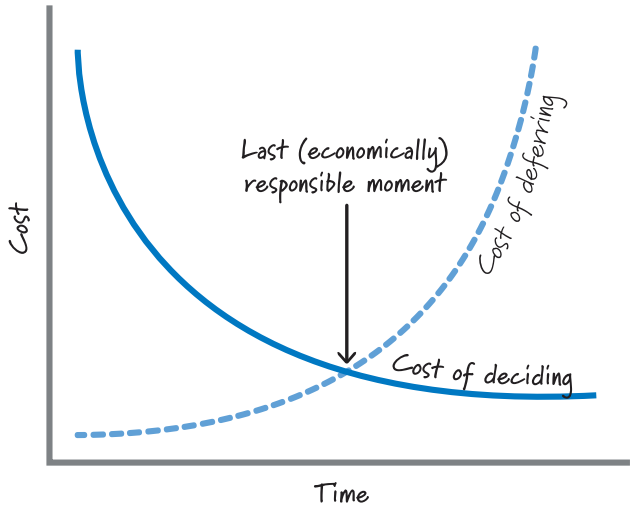



**FIGURE 3.6** Make decisions at the last responsible moment.

day or very early on? Most of us would prefer to wait until we have more information so that we can make a more informed decision. When dealing with important or irreversible decisions, if we decide too early and are wrong, we will be on the exponential part of the cost-of-deciding curve in Figure 3.6. As we acquire a better understanding regarding the decision, the cost of deciding declines (the likelihood of making a bad decision declines because of increasing market or technical certainty). That's why we should wait until we have better information before committing to a decision.

## Accept That You Can't Get It Right Up Front

Plan-driven processes not only mandate full requirements and a complete plan; they also assume that we can "get it right" up front. The reality is that it is very unlikely that we can get all of the requirements, or the detailed plans based on those requirements, correct up front. What's worse is that when the requirements do change, we have to modify the baseline requirements and plans to match the current reality (more about this in Chapter 5).

In Scrum, we acknowledge that we can't get all of the requirements or the plans right up front. In fact, we believe that trying to do so could be dangerous because we are likely missing important knowledge, leading to the creation of a large quantity of low-quality requirements (see Figure 3.7).

This figure illustrates that when using a plan-driven, sequential process, a large number of requirements are produced early on when we have the least cumulative knowledge about the product. This approach is risky, because there is an illusion that we have eliminated end uncertainty. It is also potentially very wasteful when our understanding improves or things change (as I will describe shortly).

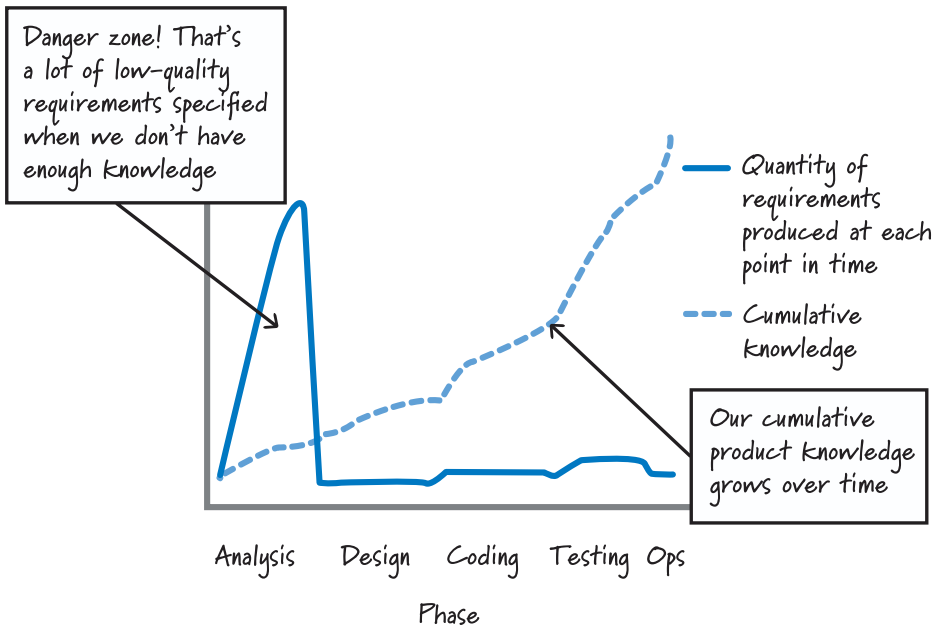



**FIGURE 3.7** Plan-driven requirements acquisition relative to product knowledge

With Scrum, we still produce some requirements and plans up front, but just sufficiently, and with the assumption that we will fill in the details of those requirements and plans as we learn more about the product we are building. After all, even if we think we're 100% certain about what to build and how to organize up front the work to build it, we will learn where we are wrong as soon as we subject our early incremental deliverables to the environment in which they must exist. At that point all of the inconvenient realities of what is really needed will drive us to make changes.

## Favor an Adaptive, Exploratory Approach

Plan-driven, sequential processes focus on using (or exploiting) what is currently known and predicting what isn't known. Scrum favors a more adaptive, trial-and-error approach based on appropriate use of exploration.

**Exploration** refers to times when we choose to gain knowledge by doing some activity, such as building a prototype, creating a proof of concept, performing a study, or conducting an experiment. In other words, when faced with uncertainty, we buy information by exploring.

Our tools and technologies significantly influence the cost of exploration. Historically software product development exploration has been expensive, a fact that favored a more predictive, try-to-get-it-right-up-front approach (see Figure 3.8).

As an example, in my freshman year at Georgia Tech (early 1980s), I (briefly) used punch cards—a tool that, like a typewriter, made you loathe to make any

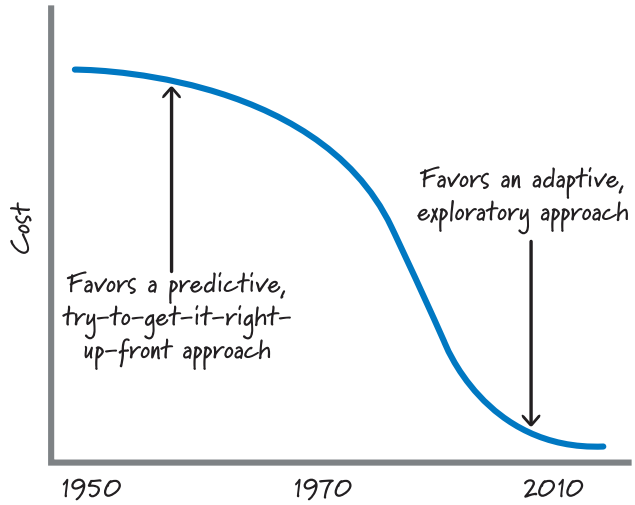



**FIGURE 3.8** Historical cost of exploration

mistakes or modifications. It was hard to embrace the concept of "Let's quickly try that out and see what happens" when, with each potential solution, you had to painstakingly create punch cards, get in the queue for the mainframe, and wait up to 24 hours to get validation of your solution. Even the cost of a simple typo was at least a day in the schedule. A waterfall-style process that allowed for careful consideration of current knowledge and prediction in the presence of uncertainty in an attempt to arrive at a good solution just made economic sense.

Fortunately, tools and technologies have gotten better and the cost of exploring has come way down. There is no longer an economic disincentive to explore. In fact, nowadays, it's often cheaper to adapt to user feedback based on building something fast than it is to invest in trying to get everything right up front. Good thing, too, because the context (the surrounding technologies) in which our solutions must exist is getting increasingly more complex.

In Scrum, if we have enough knowledge to make an informed, reasonable step forward with our solution, we advance. However, when faced with uncertainty, rather than trying to predict it away, we use low-cost exploration to buy relevant information that we can then use to make an informed, reasonable step forward with our solution. The feedback from our action will help us determine if and when we need further exploration.

## Embrace Change in an Economically Sensible Way

When using sequential development, change, as we have all learned, is substantially more expensive late than it is early on (see Figure 3.9, based on Boehm 1981).

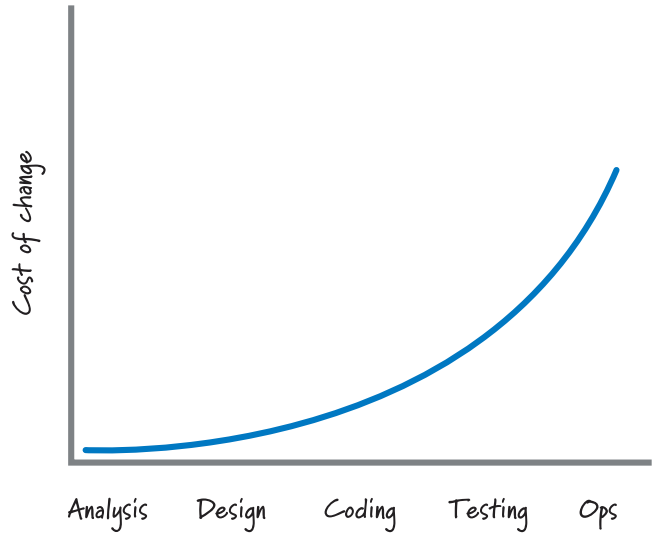



**FIGURE 3.9** Significant late cost of change with sequential development

As an example, a change made during analysis might cost $1; that same change made late during testing might cost $1,000. Why is this so? If we make a mistake during analysis and find it during analysis, it is an inexpensive fix. If that same error is not found until design, we have to fix not only the incorrect requirement, but potentially parts of our design based on the wrong requirement. This compounding of the error continues through each subsequent phase, making what might have been a small error to correct during analysis into a much larger error to correct in testing or operations.

To avoid late changes, sequential processes seek to carefully control and minimize any changing requirements or designs by improving the accuracy of the predictions about what the system needs to do or how it is supposed to do it.

Unfortunately, being excessively predictive in early-activity phases often has the opposite effect. It not only fails to eliminate change; it actually contributes to deliveries that are late and over budget as well. Why this paradoxical truth? First, the desire to eliminate expensive change forces us to overinvest in each phase—doing more work than is necessary and practical. Second, we're forced to make decisions based on important assumptions early in the process, before we have validated these assumptions with feedback from our stakeholders based on our working assets. As a result, we produce a large inventory of work products based on these assumptions. Later, this inventory will likely have to be corrected or discarded as we validate (or invalidate) our assumptions, or change happens (for example, requirements emerge or evolve), as it always will. This fits the classic pattern of a self-fulfilling prophecy (see Figure 3.10).

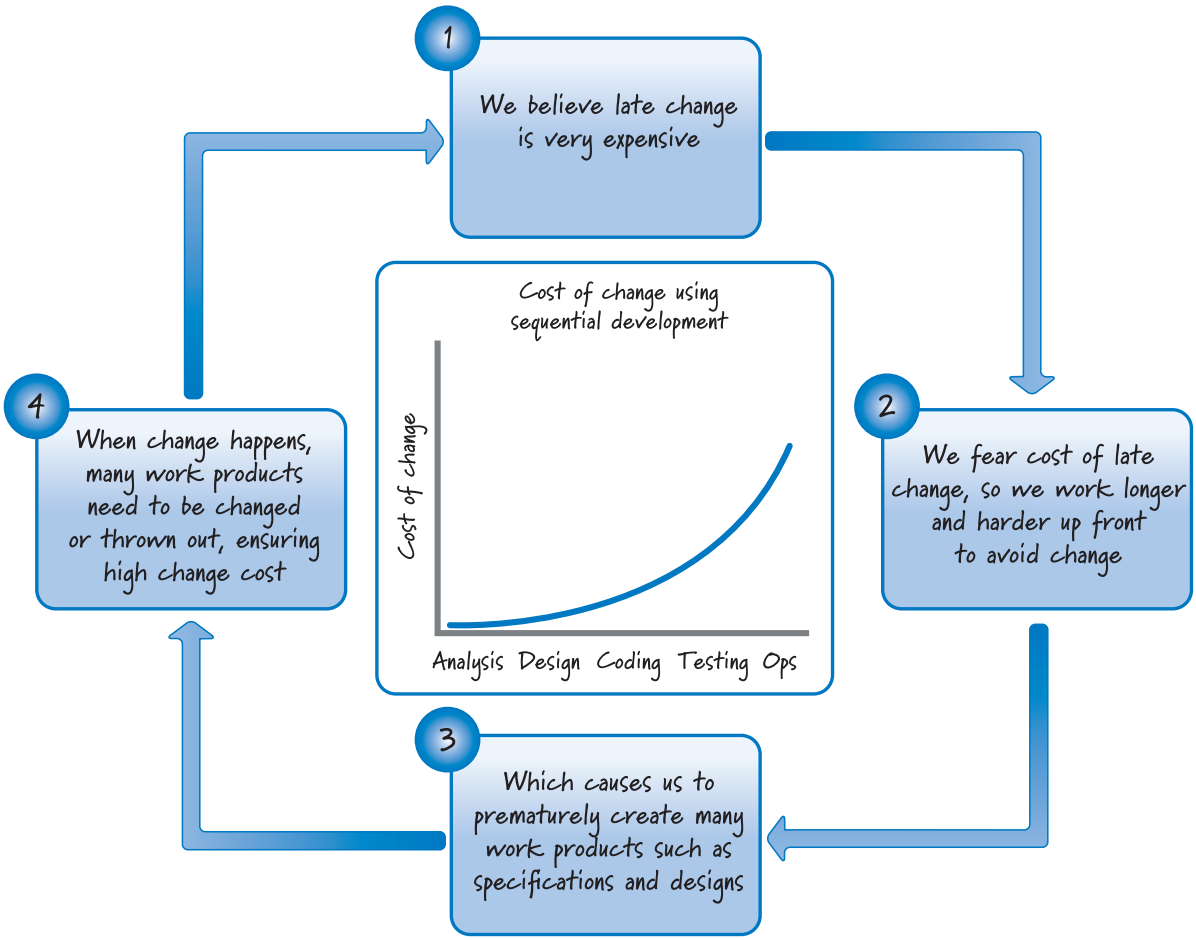



**FIGURE 3.10** Self-fulfilling prophecy

In Scrum, we assume that change is the norm. We believe that we can't predict away the inherent uncertainty that exists during product development by working longer and harder up front. Thus, we must be prepared to embrace change. And when that change occurs, we want the economics to be more appealing than with traditional development, even when the change happens later in the product development effort.

Our goal, therefore, is to keep the cost-of-change curve flat for as long as possible—making it economically sensible to embrace even late change. Figure 3.11 illustrates this idea.

We can achieve that goal by managing the amount of work in process and the flow of that work so that the cost of change when using Scrum is less affected by time than it is with sequential projects.

Regardless of which product development approach we use, we want the following relationship to be true: a small change in requirements should yield a proportionally

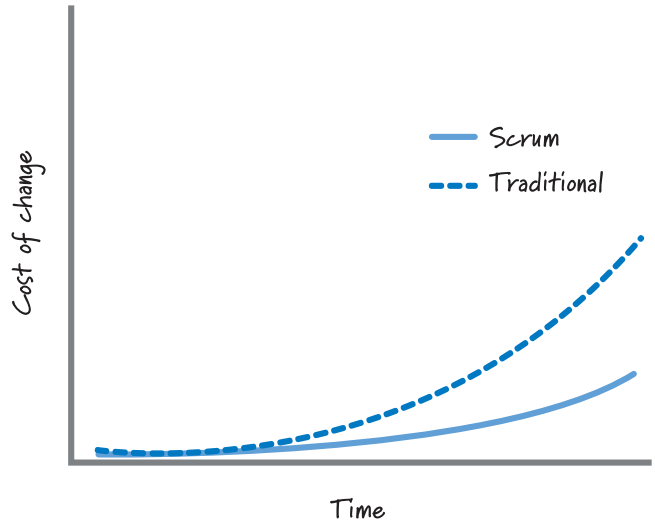



**FIGURE 3.11** Flattening the cost-of-change curve

small change in implementation and therefore in cost (obviously we would expect a larger change to cost more). Another desirable property of this relationship is that we want it to be true regardless of *when* the change request is made.

With Scrum, we produce many work products (such as detailed requirements, designs, and test cases) in a just-in-time fashion, avoiding the creation of potentially unnecessary artifacts. As a result, when a change is made, there are typically far fewer artifacts or constraining decisions based on assumptions that might be discarded or reworked, thus keeping the cost more proportional to the size of the requested change.

Using sequential development, the early creation of artifacts and push for premature decision making ultimately mean that the cost of a change rises rapidly over time as inventory grows. This causes the inflection point (where the line begins to aggressively climb up) on the traditional curve in Figure 3.11 to occur early. When developing with Scrum, there does come a time when the cost of change will no longer be proportional to the size of the request, but this point in time (as illustrated by the inflection point on the Scrum curve in Figure 3.11) occurs later.

## Balance Predictive Up-Front Work with Adaptive Just-in-Time Work

A fundamental belief of plan-driven development is that detailed up-front requirements and planning are critical and should be completed before moving on to later stages. In Scrum, we believe that up-front work should be helpful without being excessive.

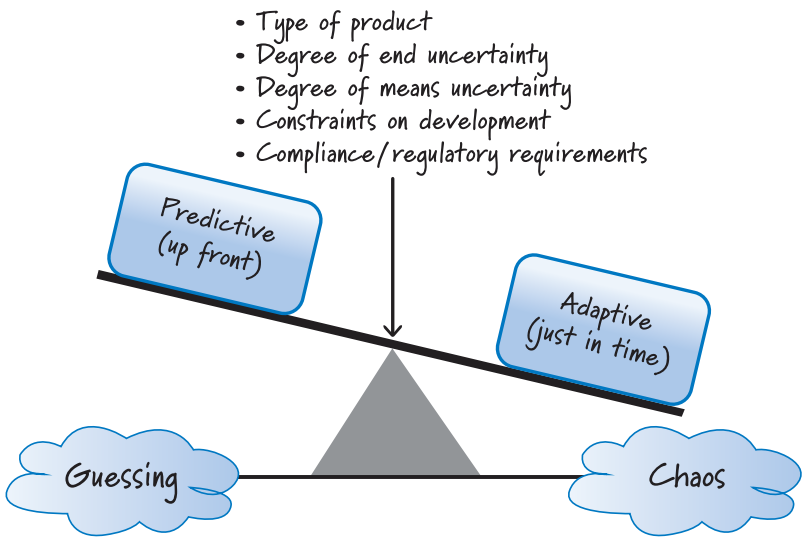



**FIGURE 3.12** Balancing predictive and adaptive work

With Scrum, we acknowledge that it is not possible to get requirements and plans precisely right up front. Does that mean we should do no requirements or planning work up front? Of course not! Scrum is about finding balance—balance between predictive up-front work and adaptive just-in-time work (see Figure 3.12, adapted from a picture by Cohn 2009).

When developing a product, the balance point should be set in an economically sensible way to maximize the amount of ongoing adaptation based on fast feedback and minimize the amount of up-front prediction, while still meeting compliance, regulatory, and/or corporate objectives.

Exactly how that balance is achieved is driven in part by the type of product being built, the degree of uncertainty that exists in both what we want to build (end uncertainty) and how we want to build it (means uncertainty), and the constraints placed on the development. Being overly predictive would require us to make many assumptions in the presence of great uncertainty. Being overly adaptive could cause us to live in a state of constant change, making our work feel inefficient and chaotic. To rapidly develop innovative products we need to operate in a space where adaptability is counterbalanced by just enough prediction to keep us from sliding into chaos. The Scrum framework operates well at this balance point of order and chaos.

## Validated Learning

When using Scrum, we organize the work to quickly create **validated learning** (a term proposed by Ries 2011). We acquire validated learning when we obtain knowledge

that confirms or refutes an assumption that we have made. I describe three agile principles related to this topic:

- Validate important assumptions fast.
- Leverage multiple concurrent learning loops.
- Organize workflow for fast feedback.

## Validate Important Assumptions Fast

An **assumption** is a guess, or belief, that is assumed to be true, real, or certain even though we have no validated learning to know that it is true. Plan-driven development is much more tolerant of long-lived assumptions than Scrum. Using plan-driven development, we produce extensive up-front requirements and plans that likely embed many important assumptions, ones that won't be validated until a much later phase of development.

Assumptions represent a significant development **risk**. In Scrum, we try to minimize the number of important assumptions that exist at any time. We also don't want to let important assumptions exist without validation for very long. The combination of iterative and incremental development along with a focus on low-cost exploration can be used to validate assumptions fast. As a result, if we make a fundamentally bad assumption when using Scrum, we will likely discover our mistake quickly and have a chance to recover from it. In plan-driven, sequential development, the same bad assumption if validated late might cause a substantial or total failure of the development effort.

## Leverage Multiple Concurrent Learning Loops

There is learning that occurs when using sequential development. However, an important form of learning happens only after features have been built, integrated, and tested, which means considerable learning occurs toward the end of the effort. Late learning provides reduced benefits because there may be insufficient time to leverage the learning or the cost to leverage it might be too high.

In Scrum, we understand that constant learning is a key to our success. When using Scrum, we identify and exploit feedback loops to increase learning. A recurring pattern in this style of product development is to make an assumption (or set a goal), build something (perform some activities), get feedback on what we built, and then use that feedback to inspect what we did relative to what we assumed. We then make adaptations to the product, process, and/or our beliefs based on what we learned (see Figure 3.13).

Scrum leverages several predefined **learning loops**. For example, the daily scrum is a daily loop and the sprint review is an iteration-level loop. I will describe these and others in subsequent chapters.